

Proving the hard-to-round search

A plan in three programs, from `htr.c` with MPFR to a search with a proved exponential

Names set in **monospace** are identifiers of `code/APaul/htr.c` or of the Rocq files in `code/APaul/rocq/`. Every number is marked **read** (off the code or a file), **measured** (a run), or **computed** (arithmetic on the other two).

1 The goal

Let x be a double and e the exponent of $\exp x$, so that $Y(x) = \exp(x) \cdot 2^{54-e}$ lies in $[2^{53}, 2^{54})$. The integers are the **round-bit grid**: the doubles and the midpoints between them. We say that x is **hard to round** at level m when

$$\text{dist}(Y(x), \mathbb{Z}) < 2^{-m},$$

that is, when the m bits after the round bit of $\exp x$ are all equal.

The theorem we want. For an interval $[x_0, x_1)$, the list of candidates the program produces contains every double of $[x_0, x_1)$ that is hard to round at level m . We ask for a **superset**: the program may report false candidates, it must not miss a true one. Removing the false ones is a separate and easier question (Section 7).

The final aim is the whole range where `exp` gives neither an infinity nor a NaN, about $x \in [-745, 709.78]$. Below $|x| < 2^{-54}$ the answer is known by hand (`exp x` rounds to 1 or just below). What is left is 64 binades on each side, $2 \cdot 64 \cdot 2^{52} = 2^{59} \approx 5.8 \cdot 10^{17}$ doubles (**computed**).

2 How the search works

The search cuts $[x_0, x_1)$ into **chunks** of $n = 2^{20}$ consecutive doubles (**read**, **search**). On a chunk starting at x , with u the ulp of x ,

$$Y(x + iu) \approx Y(x) + i \cdot Y(x)u, \quad 0 \leq i < n,$$

because the derivative of `exp` is `exp`. Only the fractional part matters, kept on 64 bits:

- **A** approximates the fractional part of $Y(x)$, times 2^{64} ;
- **B** approximates the fractional part of the slope $Y(x)u$, times 2^{64} ;
- **E** is a window: 2^{64-m} , the target, plus a bound on how far the straight line drifts from the curve over the chunk, $Y(x)u^2n^2/2$.

The inner loop adds **B** to **A** n times and reports every i where **A** is within **E** of an integer (the test $\mathbf{A} < 2 \cdot \mathbf{E}$ after shifting by **E**). This loop, a comparison and an addition, is the whole cost: 99.99% of the run on $[0.25, 0.25001)$ (**measured**, the Capla benchmark).

The one idea of the proof. The true value and the line differ by at most (drift + errors of **A**, **B**). If the window is at least target + drift + errors, no hard case falls outside it.

3 Program 1: `htr.c`, with MPFR trusted

This is the program as it is. MPFR is used in three places (**read**):

function	what it does	needed for the superset
<code>dd_exp</code>	<code>exp x</code> at 161 bits, split into three doubles $h + l + s$	yes: it seeds each chunk
<code>ref_exp</code>	<code>exp</code> at both ends, to check e is the same on the interval	yes, once per interval
<code>check</code>	decides a candidate, at $53 + m$ bits	no

We take as an axiom what MPFR documents: `mpfr_exp` returns the correctly rounded value at the requested precision.

The code, `code/APaul/htr.c`, as it is:

```
// search hard-to-round cases of exp in [0.25,0.5)
// with at least 43 identical bits after round bit

#include <stdio.h>
#include <stdint.h>
#include <mpfr.h>
#include <assert.h>
#include <math.h>

// reference code using MPFR, with RNDZ
double
ref_exp (double x)
{
    mpfr_t y;
    mpfr_exp_t emin = mpfr_get_emin ();
    mpfr_set_emin (-1073);
    mpfr_init2 (y, 53);
    mpfr_set_d (y, x, MPFR_RNDN);
    int inex = mpfr_exp (y, y, MPFR_RNDZ);
    mpfr_subnormalize (y, inex, MPFR_RNDZ);
    double ret = mpfr_get_d (y, MPFR_RNDN);
    mpfr_clear (y);
    mpfr_set_emin (emin);
    return ret;
}

// put in h+l+s a triple-double approximation of exp(x)
static void
dd_exp (double *h, double *l, double *s, double x)
{
    mpfr_t t;
    mpfr_init2 (t, 3*53+2);
    mpfr_set_d (t, x, MPFR_RNDN);
    mpfr_exp (t, t, MPFR_RNDN);
    *h = mpfr_get_d (t, MPFR_RNDN);
    mpfr_sub_d (t, t, *h, MPFR_RNDN);
    *l = mpfr_get_d (t, MPFR_RNDN);
    mpfr_sub_d (t, t, *l, MPFR_RNDN);
    *s = mpfr_get_d (t, MPFR_RNDN);
    mpfr_clear (t);
}

unsigned long checks = 0, found = 0;

static void
check (double x, int m)
{
    // printf ("check x=%la\n", x);
    checks ++;
    mpfr_t y, z;
    mpfr_init2 (y, 54);
    mpfr_init2 (z, 53 + m);
```

```

mpfr_set_d (y, x, MPFR_RNDN);
mpfr_exp (z, y, MPFR_RNDN);
mpfr_set (y, z, MPFR_RNDN);
if (mpfr_cmp (y, z) == 0) {
    printf ("%la\n", x);
    found ++;
}
mpfr_clear (y);
mpfr_clear (z);
}

// given x, -1 < x < 1, return the 64-bit value corresponding
// to the fractional part of x
static uint64_t
get_uint64 (double x)
{
    assert (-1 < x && x < 1);
    x = ldexp (x, 64);
    if (x >= 0) return x;
    uint64_t ret = (uint64_t) (-x);
    return ~ret + (uint64_t) 1;
}

// search hard-to-round cases of exp in [x0,x1)
// with at least m0 identical bits after round bit
static void
search (double x0, double x1, int m)
{
    uint64_t n = 1ul << 20;
    int e, e0, e1;

    // check ulp(x0) = ulp(nextbelow(x1))
    frexp (x0, &e0);
    frexp (nextafter (x1, x0), &e1);
    assert (e0 == e1);

    double ux = ldexp (1.0, e0 - 53); // ux = ulp(x)

    // check exp(x) lies in the same binade in [x0, x1)
    double y0 = ref_exp (x0);
    double y1 = ref_exp (nextafter (x1, x0));
    frexp (y0, &e);
    frexp (y1, &e1);
    assert (e == e1);
    double uh = ldexp (1.0, e - 53); // uh = ulp(exp(x))

    double h, l, s;
    double x = x0;
    while (x < x1) {
        dd_exp (&h, &l, &s, x);
        double dh = h * ux, dl = l * ux; // 1st derivative, multiplied by ux
        double dd = h * ux * ux; // 2nd derivative, multiplied by ux^2

        // prepare loop
        // A is the bits of h+l after the round bit
        h = ldexp (h, 54 - e); // now ulp(h) = 2
    }
}

```

```

l = ldexp (l, 54 - e);
// we now have -2 < l < 2
if (l >= 1.0) l -= 1.0;
while (l < 0) l += 1.0;
assert (0 <= l && l < 1);
s = ldexp (s, 54 - e);
uint64_t lu = get_uint64 (l), su = get_uint64 (s);
uint64_t A = lu + su;
// scale the derivatives too
dh = ldexp (dh, 54 - e);
dl = ldexp (dl, 54 - e);
dd = ldexp (dd, 54 - e);
uint64_t B = get_uint64 (dh) + get_uint64 (dl);
// the maximal error is bounded by dd/2*n^2
dd = dd / 2.0 * n * n;
// hard-to-round cases are at distance < 2^{-(m+1)} ulp(h)
uint64_t E = get_uint64 (dd) + (1ul << (64 - m));
// we want to find values of A in [-E, E]
A += E; // shift A
// now we want to find values of A in [0, 2*E]
for (uint64_t i = 0; i < n; i++) {
    if (__builtin_expect (A < 2*E, 0)) { // found potential hard-to-round case
        double xi = x + i * ux;
        check (xi, m);
    }
    A += B;
}
x += n * ux;
if (x > x1)
    n = (x1 - x) / ux;
}
}

```

```

/* With x0 = 0.25, x1 = 0.25001, m = 35
on thym.loria.fr (Intel(R) Core(TM) Ultra 7 265) with gcc 15.3.0
(-O3 -march=native):

```

```

0x1.00002385331bep-2
0x1.0000561fc7e06p-2
0x1.0001cd08ef3d4p-2
0x1.00026c769e211p-2
0x1.00028a78a7303p-2
7056503 checks, found 5 hard-to-round

```

```

real 0m39.080s
user 0m39.032s
sys 0m0.008s

```

BaCSeL:

```

make GMP=$GMP MPFR=$MPFR FPLLL=$FPLLL DEFSAL="-DEXP -DBASIS=2 -DAUTOMATIC"
zimmerma@thym:~/svn/bacsel$ ./bacsel -rnd_mode all -prec 128 -n 53 -nn 53 -m 36 -t
18 -t0 4503599627370496 -t1 4503779771355591 -d 2 -alpha 2 -e_in -2 -nthreads 4
zimmerma@thym:~/svn/bacsel$ ./bacsel -rnd_mode all -prec 128 -n 53 -nn 53 -m 36 -t
18 -t0 4503599627370496 -t1 4503779771355591 -d 2 -alpha 2 -e_in -1 -nthreads 4
./bacsel -rnd_mode all -prec 128 -n 53 -nn 53 -m 36 -t 18 -t0 4503599627370496 -t1
4503779771355591 -d 2 -alpha 2 -e_in -1 -nthreads 4

```

```

Welcome to BaCSeL version 4.0, using 4 thread(s)
Compilation flags: -DBASIS=2 -DEXP -DAUTOMATIC
LLL parameters: delta=0.999 eta=0.501
Parameters for AUTOMATIC: FAILURE_LOWERBOUND=2.0 FAILURE_UPPERBOUND=4.0
*** x=0x4.0001587f1f818p-4, distance is: 1.059463030e-11 0x1.0000561fc7e06p-2
*** x=0x4.00008e14cc6f8p-4, distance is: 1.055287224e-11 0x1.00002385331bep-2
*** x=0x4.00073423bcf5p-4, distance is: 2.577853443e-12 0x1.0001cd08ef3d4p-2
*** x=0x4.000a29e29cc0cp-4, distance is: 7.860279955e-12 0x1.00028a78a7303p-2
*** x=0x4.0009b1da78844p-4, distance is: 7.913797859e-12 0x1.00026c769e211p-2
*/
int main()
{
    double x0 = 0.25, x1 = 0.25001;
    int m = 35;
    search (x0, x1, m);
    printf ("%lu checks, found %lu hard-to-round\n", checks, found);
}

```

The obligations.

#	obligation	state
1a	$\text{dd_exp}: h + l + s - \exp x \leq 2^{-158} \exp x$ roughly, from one rounding at 161 bits and three conversions	to do, mild
1b	the conversions to 64 bits: <code>ldexp</code> , <code>get_uint64</code> , the reduction of <code>l</code> into $[0, 1)$, the wrap modulo 2^{64}	to do, fiddly
1c	the window covers target + drift + the errors of 1a and 1b	to do, the hard point
1d	the inner loop reports exactly the i in the window	done: <code>Scan.v</code>
1e	the chunks cover $[x_0, x_1)$	done at the nat layer: <code>ScanAll.v</code>

On 1c. In `htr.c`, E is target + drift and nothing else (**read**). Three errors are not in it: the curvature is taken at the start of the chunk and not its maximum over the chunk; the two truncations of B add up over n steps; and A itself is truncated. Our notes of 2026-08-01 say they fit inside the margin; this has not been re-checked. The proof decides it. If they do not fit, the fix is one term added to E .

What `htr.c` does not handle. `search` asserts things that hold on $[0.25, 0.5)$ but not on the whole range (**read**):

- `get_uint64` asserts its argument is in $(-1, 1)$. The slope $Y(x)u$ is in $[2^{e_0}, 2^{e_0+1})$ where $x \in [2^{e_0-1}, 2^{e_0})$, so it passes only for $|x| < 1/2$ (**computed**). Only the fractional part of the slope matters, so the fix is to drop the integer part first.
- $[x_0, x_1)$ must sit in one binade of x and of $\exp x$ (asserted). The full range must be cut accordingly: about $2 \cdot 64$ binades of x and a cut every $\ln 2$ for $\exp x$ (**computed**).
- Near the ends, $\exp x$ becomes subnormal (below $x \approx -708$) or overflows (above $x \approx 709.78$). The definition of “hard to round” changes for subnormal outputs.
- The last chunk overruns x_1 ; the line `n = (x1 - x) / ux` is never used. This is harmless for a superset of $[x_0, x_1)$.

4 Program 2: an abstract evaluator

Program 2 is `htr.c` with its three MPFR functions turned into parameters: `dd_exp` becomes `eval`, `ref_exp` becomes `expo`, and `check` becomes `report`. The search itself (1b to 1e) is proved once, for any `eval` and `expo` that meet a contract. It is in `code/APaul/htrplan/`.

The contract, `search.h`:

```

// Program 2: the search of htr.c with the evaluator as a parameter.

#include <stdint.h>

// eval (&h, &l, &s, x) approximates exp(x) by h + l + s. Let e be the
// exponent of exp(x) (as frexp gives it). The contract is:
// (C1) |h + l + s - exp(x)| <= 2^-kappa * exp(x)
// (C2) |l| < 2^(e-53) and |s| < 2^(e-54)
// C2 is what the search needs to read the bits after the round bit from
// l and s alone. kappa is what the proof of the window asks for.
typedef void (*eval_t) (double *h, double *l, double *s, double x);

// expo (x) returns e, the exponent of exp(x), exactly.
typedef int (*expo_t) (double x);

// report (x) is called on every candidate. The theorem to prove is that
// every double of [x0, x1) that is hard to round at level m is reported.
typedef void (*report_t) (double x, int m);

void search (eval_t eval, expo_t expo, report_t report,
             double x0, double x1, int m);

```

(C1) is the accuracy. (C2) says that l and s are small enough for the search to read the bits after the round bit from them alone: after scaling, l must be in $(-2, 2)$ and s in $(-1, 1)$ (**read**, `htr.c` lines 110 and 115). `expo` must be exact: it is only used for the check that `exp` stays in one binade, and `htr.c` gets it by rounding toward zero so that it never rounds up to the next power of 2.

The search, `search.c`. It is `htr.c`'s `search` line for line, except for the three names:

```

// Program 2: the search of htr.c, line for line, with dd_exp replaced by
// eval, ref_exp by expo, and check by report.

#include <assert.h>
#include <math.h>
#include "search.h"

// given x, -1 < x < 1, return the 64-bit value corresponding
// to the fractional part of x
static uint64_t
get_uint64 (double x)
{
    assert (-1 < x && x < 1);
    x = ldexp (x, 64);
    if (x >= 0) return x;
    uint64_t ret = (uint64_t) (-x);
    return ~ret + (uint64_t) 1;
}

// search hard-to-round cases of exp in [x0,x1)
// with at least m identical bits after round bit
void
search (eval_t eval, expo_t expo, report_t report,
        double x0, double x1, int m)
{
    uint64_t n = 1ul << 20;
    int e, e0, e1;

```

```

// check ulp(x0) = ulp(nextbelow(x1))
frexp (x0, &e0);
frexp (nextafter (x1, x0), &e1);
assert (e0 == e1);

double ux = ldexp (1.0, e0 - 53); // ux = ulp(x)

// check exp(x) lies in the same binade in [x0, x1)
e = expo (x0);
e1 = expo (nextafter (x1, x0));
assert (e == e1);

double h, l, s;
double x = x0;
while (x < x1) {
    eval (&h, &l, &s, x);
    double dh = h * ux, dl = l * ux; // 1st derivative, multiplied by ux
    double dd = h * ux * ux; // 2nd derivative, multiplied by ux^2

    // prepare loop
    // A is the bits of h+l after the round bit
    h = ldexp (h, 54 - e); // now ulp(h) = 2
    l = ldexp (l, 54 - e);
    // we now have -2 < l < 2
    if (l >= 1.0) l -= 1.0;
    while (l < 0) l += 1.0;
    assert (0 <= l && l < 1);
    s = ldexp (s, 54 - e);
    uint64_t lu = get_uint64 (l), su = get_uint64 (s);
    uint64_t A = lu + su;
    // scale the derivatives too
    dh = ldexp (dh, 54 - e);
    dl = ldexp (dl, 54 - e);
    dd = ldexp (dd, 54 - e);
    uint64_t B = get_uint64 (dh) + get_uint64 (dl);
    // the maximal error is bounded by dd/2*n^2
    dd = dd / 2.0 * n * n;
    // hard-to-round cases are at distance < 2^-(m+1) ulp(h)
    uint64_t E = get_uint64 (dd) + (1ul << (64 - m));
    // we want to find values of A in [-E, E]
    A += E; // shift A
    // now we want to find values of A in [0, 2*E]
    for (uint64_t i = 0; i < n; i++) {
        if (__builtin_expect (A < 2*E, 0)) { // found potential hard-to-round case
            double xi = x + i * ux;
            report (xi, m);
        }
        A += B;
    }
    x += n * ux;
    if (x > x1)
        n = (x1 - x) / ux;
}
}

```

Program 1 as an instance, `eval_mpfr.c`. These are `htr.c`'s three functions, unchanged except that `ref_exp` returns the exponent directly and `check` returns its verdict instead of printing it:

```
// The evaluator of Program 1 (htr.c): exp with MPFR, trusted.
// Also check, which decides a candidate; it is not part of the superset.

#include <stdio.h>
#include <mpfr.h>
#include "eval.h"

// the exponent of exp(x): htr.c's ref_exp rounds toward zero, so that the
// result never rounds up to the next power of 2
int
expo_mpfr (double x)
{
    mpfr_t y;
    mpfr_exp_t emin = mpfr_get_emin ();
    mpfr_set_emin (-1073);
    mpfr_init2 (y, 53);
    mpfr_set_d (y, x, MPFR_RNDN);
    int inex = mpfr_exp (y, y, MPFR_RNDZ);
    mpfr_subnormalize (y, inex, MPFR_RNDZ);
    int e;
    frexp (mpfr_get_d (y, MPFR_RNDN), &e);
    mpfr_clear (y);
    mpfr_set_emin (emin);
    return e;
}

// htr.c's dd_exp: exp(x) at 161 bits, split into three doubles
void
eval_mpfr (double *h, double *l, double *s, double x)
{
    mpfr_t t;
    mpfr_init2 (t, 3*53+2);
    mpfr_set_d (t, x, MPFR_RNDN);
    mpfr_exp (t, t, MPFR_RNDN);
    *h = mpfr_get_d (t, MPFR_RNDN);
    mpfr_sub_d (t, t, *h, MPFR_RNDN);
    *l = mpfr_get_d (t, MPFR_RNDN);
    mpfr_sub_d (t, t, *l, MPFR_RNDN);
    *s = mpfr_get_d (t, MPFR_RNDN);
    mpfr_clear (t);
}

// htr.c's check: is x hard to round at level m
int
check_mpfr (double x, int m)
{
    mpfr_t y, z;
    mpfr_init2 (y, 54);
    mpfr_init2 (z, 53 + m);
    mpfr_set_d (y, x, MPFR_RNDN);
    mpfr_exp (z, y, MPFR_RNDN);
    mpfr_set (y, z, MPFR_RNDN);
    int hard = mpfr_cmp (y, z) == 0;
}
```

```

mpfr_clear (y);
mpfr_clear (z);
return hard;
}

```

With them, the proof of 1c becomes: the window covers target + drift + $C(\kappa, n)$, an error term that depends only on κ and n .

How large must κ be. An error of $2^{-\kappa}$ relative is about $2^{54-\kappa}$ on the grid, so it is below one unit of A (i.e. 2^{-64}) when $\kappa \geq 118$ (**computed**). With $\kappa \approx 120$, the evaluator errors are small next to the drift, which is about 2^{48} units on $[0.25, 0.25001)$ (**computed** from `htr.c`'s formula). MPFR at 161 bits gives $\kappa \approx 158$, much more than needed.

What this buys. Program 1 is an instance, and its contract follows from the MPFR axiom (1a). Program 3 is another instance, with no axiom.

5 Program 3: a proved evaluator

Now `eval` is a program proved against the contract. The first question is how fast it must be, and the answer is: not very.

Speed. `eval` is called once per chunk. The inner loop runs about $1.5 \cdot 10^9$ steps a second (**measured**, one P-core, turbo off), so a chunk takes about 0.7 ms (**computed**). An evaluator of 10 μs would add 1.4% (**computed**). On the full range there are $2^{59}/2^{20} = 2^{39} \approx 5.5 \cdot 10^{11}$ chunks (**computed**), and the whole search is about $3.9 \cdot 10^8$ core-seconds, 12 core-years (**computed**), whatever the evaluator. So the choice of evaluator is a question of **proof effort**, not of speed.

Three possible evaluators.

evaluator	how	certificates	difficulty
Tang, fixed point	$x = k \ln 2 + j/256 + r$; a table of 256 values $\exp(j/256)$; one polynomial for $\exp r$, $ r \leq 2^{-9.5}$: degree 10 gives 2^{-130} , degree 13 gives 2^{-169} (computed from the Taylor bound); integers only	1 polynomial, 256 table entries, 1 reduction	3
pieces + section 5	one certified polynomial per piece, like <code>Cheb.v</code> ; the chunk seeds by exact additions (<code>Shift.v</code> , <code>Pdir_exp</code>)	$6 \cdot 10^5$ at degree 7, $4 \cdot 10^3$ at degree 15, for 2^{-100} (computed , formula checked at one point)	2 each, but volume
short Taylor near 0	for small $ x $, $\exp x = 1 + x + \dots$ with few terms, no table	1 polynomial per binade group	1–2

Recommendation. Tang for the bulk of the range: one proof, independent of the range. A different evaluator may be used where it makes the proof simpler, for instance the short Taylor series near 0, where no reduction is needed. Since speed does not depend on the evaluator, mixing them is only worth it for the proof.

The code. Program 3 is written, as `eval_fix.c`, and plugs into the same `search.c`. It uses GMP's integers (`mpz`) and nothing else: no MPFR, no floating point beyond reading x and writing h, l, s . It takes degree 13 and 200 bits after the point, so that its accuracy is that of MPFR's 161 bits and the two can be compared candidate by candidate. Degree 10 would be enough for $\kappa = 120$.

```

// The parameters of eval_fix.c.

#include <gmp.h>

#define FIX_P 200 // bits after the point of every fixed-point number
#define FIX_TAB 256 // table size:  $x = (256k + j) \ln 2 / 256 + r$ 
#define FIX_LG 32 // extra bits of  $\ln 2 / 256$ , so that  $N * \ln 2 / 256$  is exact
// to  $2^{-\text{FIX\_P}}$  for  $|N| < 2^{32}$ 
#define FIX_DEG 13 // degree of the Taylor polynomial of  $\exp(r)$ 

// The evaluator of Program 3:  $\exp$  with integers only (GMP's mpz), to be
// proved. Tang's scheme:
//  $x = N \ln 2 / 256 + r$ ,  $N = 256k + j$ ,  $0 \leq j < 256$ ,  $|r| \leq 2^{-9.5}$ 
//  $\exp(x) = 2^k * 2^{(j/256)} * \exp(r)$ 
// A fixed-point number is an integer  $X$  standing for  $X / 2^{\text{FIX\_P}}$ .
// Designed error (not yet proved):  $2^{-169}$  on  $y$ ,  $2^{-158}$  after the split.

#include <assert.h>
#include "fix.h"
#include "eval.h"
#include "fix_table.h"

static mpz_t T[FIX_TAB]; //  $2^{(j/256)}$ , rounded to nearest
static mpz_t ln2; //  $\ln 2 / 256$ , with FIX_LG extra bits
static mpz_t C[FIX_DEG + 1]; //  $1/i!$ , rounded down
static mpz_t rmax; // bound on  $|r|$ :  $181/256 * 2^{-9} < 2^{-9.5}$ 
static mpz_t err; // bound on the error of  $y$ :  $2^{-160}$ 

void
fix_init (void)
{
    for (int j = 0; j < FIX_TAB; j++)
        mpz_init_set_str (T[j], fix_T[j], 16);
    mpz_init_set_str (ln2, fix_ln2, 16);
    mpz_t f;
    mpz_init_set_ui (f, 1);
    for (int i = 0; i <= FIX_DEG; i++) {
        if (i > 0) mpz_mul_ui (f, f, i);
        mpz_init (C[i]);
        mpz_setbit (C[i], FIX_P);
        mpz_fdiv_q (C[i], C[i], f);
    }
    mpz_clear (f);
    mpz_init_set_ui (rmax, 181);
    mpz_mul_2exp (rmax, rmax, FIX_P - 17);
    mpz_init (err);
    mpz_setbit (err, FIX_P - 160);
}

//  $\exp(x) = y / 2^{\text{FIX\_P}} * 2^k$ ; returns k
static long
fix_exp (mpz_t y, double x)
{
    mpz_t X, r, t;
    mpz_inits (X, r, t, NULL);

```

```

// X = x 2^FIX_P, exactly
int ex;
double mx = frexp (x, &ex);
long sh = ex - 53 + FIX_P;
assert (sh >= 0);
mpz_set_si (X, (long) ldexp (mx, 53));
mpz_mul_2exp (X, X, sh);
// r = x - N ln2/256, checked small
long N = (long) nearbyint (x * (FIX_TAB / M_LN2));
mpz_mul_si (t, ln2, N);
mpz_fdiv_q_2exp (t, t, FIX_LG);
mpz_sub (r, X, t);
mpz_abs (t, r);
assert (mpz_cmp (t, rmax) < 0);
// exp(r) by Horner's rule, one truncation per product
mpz_set (y, C[FIX_DEG]);
for (int i = FIX_DEG - 1; i >= 0; i--) {
    mpz_mul (y, y, r);
    mpz_fdiv_q_2exp (y, y, FIX_P);
    mpz_add (y, y, C[i]);
}
// times 2^(j/256)
long j = ((N % FIX_TAB) + FIX_TAB) % FIX_TAB;
mpz_mul (y, y, T[j]);
mpz_fdiv_q_2exp (y, y, FIX_P);
// y is not within err of a power of 2, so its exponent is that of exp(x)
mpz_sub (t, y, err);
size_t nb = mpz_sizeinbase (t, 2);
mpz_add (t, y, err);
assert (mpz_sizeinbase (t, 2) == nb);
mpz_clears (X, r, t, NULL);
return (N - j) / FIX_TAB;
}

int
expo_fix (double x)
{
    mpz_t y;
    mpz_init (y);
    long k = fix_exp (y, x);
    int e = (int) mpz_sizeinbase (y, 2) - FIX_P + k;
    mpz_clear (y);
    return e;
}

// the top 53 bits of y, as a double scaled by 2^sc; y keeps the rest
static double
top53 (mpz_t y, long sc)
{
    if (mpz_sgn (y) == 0) return 0.0;
    long sh = (long) mpz_sizeinbase (y, 2) - 53;
    if (sh < 0) sh = 0;
    mpz_t hi;
    mpz_init (hi);
    mpz_fdiv_q_2exp (hi, y, sh);
    double d = ldexp (mpz_get_d (hi), sh + sc);

```

```

    mpz_mul_2exp (hi, hi, sh);
    mpz_sub (y, y, hi);
    mpz_clear (hi);
    return d;
}

void
eval_fix (double *h, double *l, double *s, double x)
{
    mpz_t y;
    mpz_init (y);
    long k = fix_exp (y, x);
    *h = top53 (y, k - FIX_P);
    *l = top53 (y, k - FIX_P);
    *s = top53 (y, k - FIX_P);
    mpz_clear (y);
}

```

The constants, the 256 values $2^{j/256}$ and $\ln 2/256$, are in `fix_table.h`, produced with MPFR by `gen_fix.c`. That producer is not trusted: each constant is to be certified once in Rocq. The coefficients $1/i!$ are computed exactly from integers and need no certificate. The two `asserts` are part of the design: one checks $|r| \leq 2^{-9.5}$, the other that y is not within its error of a power of 2, so that its exponent is the exponent of $\exp x$. Proving the program means proving that the rest computes within the bound, not that the asserts never fire.

The Tang obligations.

part	difficulty
the polynomial on r : one <code>interval</code> call, like <code>cheb_valid</code>	1
the 256 table entries: one small interval check each	1
the reduction: $\ln 2$ split on several words, $ k \leq 1075$	3
the sum of the truncation errors, one per product	2-3

6 The test

`make test` in `code/APaul/htrplan/` runs the three programs on the same slice and compares their output line by line: every candidate, then the hard cases found. Program 1 is `htr.c` itself, with its interval and m taken from the command line by a `sed` (`htr_slice.c`). All numbers here are **measured**, 2026-09-28, on this desktop:

slice	chunks	candidates	result
$[0.25, 0.25 + 10^4 \cdot 2^{-34})$	10 000	410 325	identical in all three; one hard case, <code>0x1.00002385331bep-2</code> , the first of <code>htr.c</code> 's five
$[-0.25 - 10^4 \cdot 2^{-34}, -0.25)$	10 000	498 078	identical in all three; one hard case

The two evaluators were also compared directly with MPFR at 400 bits on ten points between -700.3 and 700.1 (all but one, see below): Program 3's relative error is between 2^{-159} and 2^{-188} (**measured**). The `h` of the two programs is not always the same (Program 3 truncates where MPFR rounds); the contract does not ask it to be.

One limit found by the test. At $x = -700.3$ the error of both evaluators is $2^{-66.5}$ (**measured**). There $\exp x \approx 2^{-1010}$, so l and s fall below the smallest double and are lost. Three doubles carry 159

bits only while $\exp x \geq 2^{-1074+159} = 2^{-915}$, that is $x \geq -634$ (**computed**). Below that the interface itself must change, for instance by passing the fixed-point value to the search directly instead of three doubles.

7 Removing the false candidates

The superset needs no **check**. To get the exact list, each candidate is evaluated once at about 2^{-100} and decided. With Program 3 this is the same evaluator. The number of candidates matters here: about 41 per chunk on $[0.25, 0.25001)$ (**measured**: 7 056 503 over 171 799 chunks). The count comes from the width of the window, $2E/2^{64} \approx 2^{-14.6}$ per double (**computed**). A smaller n shrinks the drift quadratically and so the candidates, at the price of more chunks.

8 What to do, in order

1. **Program 1, 1a to 1c**, on $[0.25, 0.25001)$ where 1d and 1e are done. This settles the hard point, the window, with MPFR as the only axiom.
2. **Program 2**: redo 1c against the contract of `search.h`, for any `eval`. The code is written and tested (Section 6).
3. **Generalise the search** to the whole range: the slope above 1, the cuts at binades, the subnormal and overflow ends.
4. **Program 3**: the proof of `eval_fix.c` against the contract, and the certificates of its constants. The code is written and tested (Section 6). Add the short Taylor series near 0 if it simplifies things.
5. **The link to the running program**. The proofs above are about a Rocq model of the search. Tying them to the code that runs is a separate step; with Capla (our port of `htr_plain.c`, `code/APaul/capla/`) the program has a formal semantics, but how a Capla program is linked to a Rocq proof has not been tried.